Lab LMS Database Schema v4

Changes, Additions and Definitions

Astrid and Christian Brinkmann — 19.07.2025

What do we need? - new Tables to add to the database

- **checkpoints** table that contains all checkpoints of all challenges
  - **challenge_id**: serial
    - *unique* and *index* on this field
  - **fixed_order**: bool
    - If true, the order of sections is fixed and cannot be changed automatically by the system.
    - If false, the order can be changed.
  - **duration**: int8
    - The duration specified in the checkpoint configuration view in minutes
  - **status**: int2
    - Shows in what stage the checkpoint is- referring to the states Upcoming, Current, Pending Grading ... in the teacher view
    - *index* on this field
  - **created_at**: timestamptz
    - The time when the checkpoint was created. -- **author_id**: int4
    - The author of the checkpoint.
    - This could be a foreign key to a users table or similar.
  - **version**: int4
    - can use later if we want to support version control The pk is (**challenge_id**, **version**)
- **sections** table that contains all sections of all checkpoints
  - **challenge_id**: int4
    - The challenge (which should be a checkpoint) this section belongs to.
    - *key* to the **challenges** table
    - *index* on this field
  - **checkpoint_version**: int4
    - The version of the checkpoint this section belongs to.
    - *key* together with **challenge_id** to the **checkpoints** table
    - *index* on this field
  - **id**: serial
    - *unique* and *index* on this field
  - **name**: varchar

    * Essentially the title of the section.

  - **section_text**: varchar
    * The text content of the section that is displayed to the student. Can potentially refer to files in the **section_files** table to display images, videos, or other media.

  - **fixed_order**: bool
    * If true, the order of questions is fixed and cannot be changed automatically by the system.

    * If false, the order can be changed.

  - **position**: int2
    * Defines the order of sections - if the order is fixed

    * Lower values come first.

- **section_files** table that contains all files related to sections
  - **section_id**: int4
    * *key* to the **sections** table

    * *index* on this field

  - **id**: serial
    * *unique* and *index* on this field

  - **name**: varchar
    * The actual human readable name of the file.

  - **key**: varchar
    * The key used in aws s3 to access the file.

- **question_files** table that contains all files related to questions
  - **question_id**: int8
    * *key* to the **checkpoint_questions** table

    * *index* on this field

  - **id**: serial
    * *unique* and *index* on this field

  - **name**: varchar
    * The actual human readable name of the file.

  - **key**: varchar
    * The key used in aws s3 to access the file.

- **checkpoint_question_slots** table that contains all slots for questions in checkpoints, associates questions with their sections
  - **section_id**: int4
    * *key* to the **sections** table

    * *index* on this field

  - **id**: bigserial
    * *unique* and *index* on this field
  - **max_mark_numerator** *int8*
  - **max_mark_denominator** *int8*
    * The maximum number of points for the question slot expressed as a rational number.
    * This allows for more precise grading, as we can represent marks like 1/3 without losing precision.
  - **position**: int2
    * Defines order of the question in its section - if the order is fixed
    * Lower values come first.
    * This allows for reordering of questions in a checkpoint.

- **checkpoint_questions** table that contains all questions related to checkpoints (not races or sprints)
  - **id**: bigserial
    * *unique* and *index* on this field
    * Ordinariliy, ~4 billion questions should be more than sufficient, but automatically generating them could throw a wrench in the works if something goes wrong.
    * Using int8 allows for 18 quintillion questions.
  - **question_slot_id**: int4
    * The slot this question belongs to.
    * *key* to the **checkpoint_question_slot** table
    * *index* on this field
  - **content**: jsonb
    * The content of the question.
    * This could be a JSON object containing the question text, selectable answers if applicable, the correct/reference answer(s), Explanation, and any other relevant information.
    * *index* on this field
  - **answer_type**: int4
    * Single choice, multiple choice, open ended, cloze, maybe more in the future
  - **author_id**: integer
    * user who created the question
    * *key* to the **users** table
    * *index* on this field

- **question_tags** table that allows for tagging questions
  - **id**: serial
    * *unique* and *index* on this field

        * The tag this question belongs to.

    - **question_id**: int8
        * *index* on this field
        * The question this tag belongs to.
        * *key* to the **checkpoint_questions** table

    - **tag**: varchar
        * *index* on this field
        * The tag itself.
        * This could be a string representing the tag name or identifier.

    - **tag_details**: jsonb
        * Additional details about the tag.
        * This could be a JSON object containing metadata about the tag, such as its author, creation date, or categories et cetera.

- **question_slot_selections** table that specifies which questions are selected for progress
    - **question_slot_id**: int4
        * *index* on this field
        * The slot this question belongs to.
        * *key* to the **checkpoint_question_slot** table

    - **question_id**: int8
        * *index* on this field
        * The question this slot belongs to.
        * *key* to the **checkpoint_questions** table

    - **progress_id**: int4
        * *index* on this field
        * The progress the selection belongs to.
        * *key* to the **progresses** table

- **prompts** table that contains prompts we use to interact with llms
    - **id**: serial
        * *unique* and *index* on this field
        * The prompt this question belongs to.

    - **purpose_id**: int2
        * *index* on this field

    * The use case of the prompt, like “generate answers”, “generate questions”, “generate explanations”, “check answers”, “check questions”, “check explanations”, “generate code”, “check code”, “generate tests”, “check tests” et cetera.
      · But encoded as an int2
    * Specifies the purpose of the prompt.
  - **author_id**: int4
    * The author of the prompt, a user id.
    * *index* on this field
    * *key* to the **users** table
  - **content**: jsonb
    * The content of the prompt.
    * This could be a JSON object containing the prompt text, parameters, and any other relevant information.

- **comments** table that contains comments
  - **id**: int8
    * *unique* and *index* on this field
  - **progress_id**: int4
    * *index* on this field
    * the progress the comment belongs to
    * *key* to the **progresses** table
  - **question_id**: int8
    * *index* on this field
    * *key* to the **checkpoint_questions** table
    * The question this comment belongs to.
    * This allows for associating comments with specific questions in a checkpoint.
  - **content**: varchar
    * content of the comment
  - **user_id**: int4
    * user that wrote the comment
  - **mentor**: boolean
    * true if the user who wrote the comment is the teacher, false otherwise
  - **updated_at**: timestamptz
  - **created_at**: timestamptz

- **progress_mentor_interactions** table that specifies for a progress whether the grade given by the mentior is visible for the team belonging to the progress (in our case team with one member) and also used to save if a

teacher allows the student to take more time for the test

  - **progress_id**: int4
    * *unique* and *index* on this field
    * *key* to the **progresses** table
  - **published_status**: int4
    * 0: teacher review not published
    * 1: teacher review published
    * 2: teacher review published and reviewed by student
    * 3: review window closed
  - **exam_started_at**: timestampz
    * time when the exam was started by the student
  - **auto_eval_started_at**: timestampz
    * time when the auto-evaluation was started by the student
      · we can use this to get the evaluation time by subtracting this from **auto_eval_finished_at**
      · we can also know that this evaluation is currently ongoing and don't start multiple evaluations at the same time
  - **auto_eval_finished_at**: timestampz
    * time when the auto-evaluation was finished by the student
      · we can use this to get the evaluation time by subtracting **auto_eval_started_at** from this
      · if this is not set but **auto_eval_started_at** is, we know that the evaluation is currently ongoing and don't start multiple evaluations at the same time. We can also define a timeout, so the evaluation could be restarted if it takes too long.
  - **time_extension**: minutes
    * null -> no extension
    * otherwise: number of minutes the student is allowed to take in addition for the checkpoint
  - **extension_start_at**: timestamptz
    * null -> no extension
    * otherwise: time when the extended time was started by the student
      · if the exam was finished and got extended afterwards, this time will be set to when the student restarts the exam, such that he or she has to be finished by **extension_start_at** + **time_extension**
      · if the exam is still ongoing, this time will be set to **start_at** + **duration**
  - **comments_finished**: boolean
    * did the teacher finish the conversation

- **progress_checkpoint_versions** table that specifies the checkpoint version for a progress
  - **progress_id**: int4
    * *unique* and *index* on this field

    * *key* to the **progresses** table

  - **checkpoint_version**: int4
    * *index* on this field
    * *key* together with **checkpoint_id** from **progresses** to the **checkpoints** table
    * The version of the checkpoint this progress belongs to.
    * This allows for tracking which version of a checkpoint a progress is based on.

What do we reuse?

- **challenges** table that contains all the challenges (also Races and sprints)
  - **id**: serial
  - **name**:varchar
  - **summary**:varchar
    * text summarizing the challenge
  - **type_id**:int4
    * SPRINT: 1, RACE: 2, CHECKPOINT: 3
  - **branch_id**: int4
    * the branch the challenge belongs to
  - **max_mark**: int4
    * in races and sprints: maximum 200
  - **estimation**: int4
    * probably estimation of how long the challenge should take (not 100% sure)
    * default value 1
    * can pobably ignore
  - **upload_type_id**: int4 -GIT: 1, FILES: 2 -default 2
  - **assessment_number**:int4 -default when creating the challenge is 0 -number of assessments by other students for the challenge -we can ignore this
  - **team_size**: int4
    * if type_id!=CHALLENGE_TYPE.Race: default=1
    * we create a team containing one member for each exam for consistency with the other type of challenges
  - **auto_team**: boolean
    * can ignore
  - **allowed_mimetypes**: jsonb
    * the kind of files the students are allowed to upload for their solutions (Docs, Table, Archive, Images)
    * can ignore
  - **start_at**:timestamptz

  - **end_at**:timestamptz

  - **dependeny**: integer
    * CHAIN: 1, TIME: 2, TIME_CHAIN: 3

    * CHAIN: possibly other challenges have to be completed before

    * TIME: start and end time are specified

    * TIME_CHAIN: TIME + CHAIN

  - **fos_id**: integer
    * field of studies challenge belongs to

  - **has_tests**: boolean
    * specifies whether there are unit tests

    * default: false -> can ignore

- **progresses** when a student accepts a challenge, a progress is created to store information about the execution of the challenge (see in the code accept.js/environment.js), in our case of the checkpoints this stores information about an exam
  - **id**: serial

  - **mark**: int4
    * used for challenges with assessments to store the average mark for the assessments

  - **challenge_id**: int4 -challenge the progress belongs to

  - **graded_at**: timestamptz
    * time when the mark was set for the assessments in case there are assessment

    * also used for challenges without assessments to store times for transition to a certain status

  - **status**: int
    * PENDING: -1, IN_PROGRESS: 0, REFLECTION: 1, ASSESSMENT: 2, FINISHED: 3, GIVE_UP: -2, RETRY: -3,

  - **createdAt**: timestamptz

  - **updatedAt**: timestamptz

  - **team_id**: int4
    * we create a team with only one user which is the student taking the exam, the team_id is a reference to the key in the teams table

  - **start_at**:timestamptz
    * time when the progress was created, progress is create for the student when he/she accepts the challenge

  - **ready_to_check**:boolean
    * indicates whether the challenge can be assessed by other students

    * we can ignore this

  - **finished_at**:timestamptz
    * time students submits solution (see finish.js)

- **teams**: when a challenge is accepted a team is created for the progress, for the checkpoints we create a team with only one member who is the leader of the team (see code createProgressEnv->createTeam)
  - **id**: int4
  - **locked**: boolean
    * true -> the team cannot be changed anymore
  - **leader_id**: int4
    * foreign key referencing users.id

- **team_members**:
  - **user_id**: int4
    * foreign key referencing users.id
  - **team_id**: int4
    * foreign key referencing teams.id
  - **pending**: boolean

- **mentor_marks**
  - **progress_id**: int4
    * progress (in our understanding exam) that is marked
  - **mark**: int4
    * we can use this to save the points across the exam the student got
  - **mentor_id**: int4
  - **graded_at**: timestamptz
  - **comment**: text

- **checkpoint_answers**
  - **progress_id**: int4
    * *key* to the **progresses** table
    * *index* on this field
    * The progress (exam) this answer belongs to.
    * the progress we want to refer to
  - **question_id**: int8
    * *key* to the **checkpoint_questions** table
    * references question for which the answer is saved
  - **answer**: jsonb
    * used to save the answers given by the student for the exam and also later for the teacher to save how

answers from the student are graded

- **challenges_files** table that contains all files related to challenges
  - **challenge_id**: int4
  - **id**: serial
    - *unique* and *index* on this field
  - **name**: varchar
    - The actual human readable name of the file.
  - **key**: varchar
    - The key used in aws s3 to access the file.

- **mentor_activities**
  - **id**: int8
    - *unique* and *index* on this field
  - **user_id**: int4
    - *key* to the **users** table
    - *index* on this field
    - The user who performed the activity
  - **activity**: varchar
    - The activity performed by the user
    - This could be a string representing the activity name or identifier.
  - **activity_data**: jsonb
    - Additional data related to the activity.
    - This could be a JSON object containing metadata about the activity, such as timestamps, parameters, or other relevant information.
  - **affected_challenge**: int
    - *index* on this field
    - The challenge affected by the activity.
    - This could be a foreign key to a challenges table or similar.
  - **date**: timestampz
    - *index* on this field
    - The date and time when the activity was performed.
    - This could be a timestamp with timezone information to ensure accurate time representation across different time zones.

activity : varchar affected_project : int affected_challenge : int date : timestampz

- **challenge_subjects** table that specifies the subjects for the challenges, we should do the same that is done for the other challenges to store the subjects (see function updateChallenge in the code)

  - **challenge_id**: int4
  - **subject_id**: int4
    - foreign key references subjects.id
  - **percentage**: int4
  - **topics**: text

- **subjects** table that specifies the subjects that exist
  - **id**: int4
  - **name**: text

Changes and additions as a diagram

The r10 schema diagram is not included in this export.

On the left side are existing tables that are not changed, on the right side are new tables or existing tables that are changed.

JSON Schemas

checkpoint_questions

The `answer_type` field can be one of the following values:

- `1`: Single-choice question
- `2`: Multiple-choice question
- `3`: Cloze question (fill-in-the-blank)
- `4`: Open-ended question

single-choice

```
{
    "question_text": "What is the capital of France?",
    "options": [
        "Paris",
        "London",
        "Berlin",
        "Madrid"
    ],
    "correct_answer": 0,
    "explanation": "London is the capital of the United Kingdom, Berlin is the capital of Germany, and Madi
}
```

multiple-choice

```
{
    "question_text": "Which of the following are programming languages?",
    "options": [
        "Python",
        "HTML",
```

```
        "CSS",
        "JavaScript"
    ],
    "correct_answers": [0, 3],
    "explanation": "Python and JavaScript are programming languages, while HTML and CSS are markup and styl
}
```

cloze questions aka fill-in-the-blank aka "lückentext"

```
{
    "question_text": [
        "The ____ model is a seven-layer framework that standardizes ____ communication, from ____ transmis
    ],
    "correct_answers": ["OSI", "network", "physical data", "application-level", "devices", "TCP/IP"],
    "incorrect_answers": ["data link", "transport", "UDP"],
    "explanation": "The OSI model is a conceptual framework used to understand and standardize the functio

    The seven layers are:
    1. Physical Layer
    2. Data Link Layer
    3. Network Layer
    4. Transport Layer
    5. Session Layer
    6. Presentation Layer
    7. Application Layer

    Nowadays, the OSI model is often compared to the more commonly used TCP/IP model, which has fewer layer
}
```

open-ended

```
{
    "question_text": "Explain the concept of polymorphism in object-oriented programming.",
    "correct_answer": "Polymorphism is the ability of different classes to be treated as instances of the s
    "explanation": "Polymorphism is a core concept in object-oriented programming that enables flexibility
}
```

checkpoint_answers

Single-choice:

```
{
    "answer": 0
}
```

Results in $\frac{1}{1} = 100\%$ of the max_marks for the question.

Note that we are using `-1` to indicate no answer being selected.

Multiple-choice:

```
{
    "answer": [0, 2],
}
```

Results in $\frac{1}{2} = 50\%$ of the max_marks for the question.

Note that we are using `[-1]` to indicate no answer being selected, although this is not strictly necessary, as the answer can also be an empty array, but this is more consistent with the single-choice answers.

Fill-in-the-blank aka "Lückentext":

```
{
    "answer": [
        ["TCP/IP", "", "physical data", "application-level", "devices", "OSI"],
    ]
}
```

Results in $\frac{3}{6} = 50\%$ of the max_marks for the question.

Note: We are using "" for no answer for the fill-in-the-blank questions.

Open-ended:

```
{
    "answer": "No, writing the requirements is no fun."
}
```

Results in $\frac{0}{1} = 0\%$ of the max_marks for the question.

Changes to existing tables

- **checkpoint_questions** table
  - remove **challenge_id** column
    - This is not needed anymore, as the questions are now associated with checkpoints through the **checkpoint_question_slots** table, which in turn is associated with sections of checkpoints.
  - remove **variant**
    - We do not need this.
  - remove **answer_config** column
    - This is not needed anymore, as the answer configuration is part of the **content** field.
  - change type of **id** column to *bigserial*
    - This allows for more questions in the future, as we can have up to 18 quintillion questions.
  - remove **points** column (we use **max_marks** for the question_slot instead)
- **checkpoint_answers** table
  - add **mark_numerator** *int8*
  - add **mark_denominator** *int8*
  - add **mentor_mark_numerator** *int8*
  - add **mentor_mark_denominator** *int8*
    - This allows us to store the mentor's mark as a rational number, which can be useful for more precise grading.
    - The numerator and denominator can be used to calculate the final mark, this way we can represent even marks like 1/3 without losing precision.
  - add **evaluation_text** *varchar*
    - This allows the mentor to provide an explanation for the mark given. Also allows an LLM in the auto-evaluation to provide an explanation for the mark given.
  - add **evaluation_flag** int4

        * This allows an LLM to flag various issues with an answer, such as perhaps
            - the answer might be ai-generated
            - the answer is identical to an answer of another student
            - the answer contains instructions or inappropriate content etc.
    - add **updated_at** *timestamptz*
        * This allows us to track when the answer was last updated.

Notes

- Serial and Bigserial are just auto-incrementing integer types in PostgreSQL.
    - Serial is a 4-byte integer (int4), while Bigserial is an 8-byte integer (int8).
    - If we expect a large number of entries, we should use Bigserial for the IDs.
    - For example, if we (theoretically) expect to have more than 2 billion entries in the **checkpoint_questions** table, we should use Bigserial for the question_id.
- Apparently, postgresql can do some nice indexing shenanigans with jsonb fields, so we can use jsonb for the content field in the **checkpoint_questions** table.
    - This allows us to store complex data structures in a single field and still be able to query them efficiently.
    - However, this may not be the best choice for all use cases, as it can make querying more complex and less efficient.
- Maybe sections should also store marks ... but that's not strictly necessary